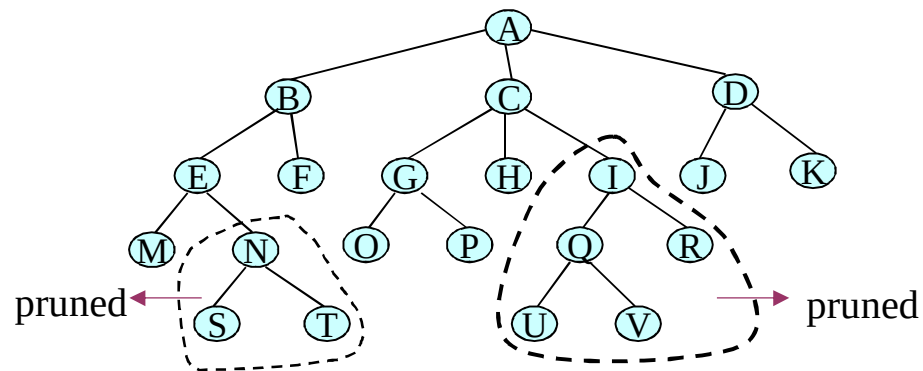

Preventing Overfitting

- ▶ **Pre-pruning:** stop growing the tree when data split is not statistically significant
 - ▶ For example,
 - ▶ set a threshold $\alpha > 0$ and declare a node terminal if percentage of examples in the most common class $> \alpha$, or
 - ▶ set a threshold $\beta > 0$ and declare a node terminal if highest information gain $< \beta$
 - ▶ Problems:
 - ▶ Hard to set the threshold value
 - ▶ The splitting is either stopped too soon or continued too far depending on the threshold
 - ▶ Using more complicated stopping rule does not help

Preventing Overfitting (Cont'd)

- ▶ **Post-pruning:** grow full tree, allowing it to overfit the data, and then remove some subtrees



- ▶ More successful in practice
- ▶ Criterion is needed to determine what to prune

Post-pruning Functions

- ▶ Post-pruning functions are needed to determine which part(s) of the tree should be pruned
- ▶ Several post-pruning functions, including:
 - ▶ Reduced-error (Quinlan, 83)
 - ▶ Error-complexity (CART, 84)
 - ▶ Pessimistic error (Quinlan, 86)
 - ▶ Minimum description length (MDL) (SLIQ by Mehta *et al.* 1996)
 - ▶ Minimum error (Niblett & Bratko, 86)
 - ▶ Critical value (Mingers, 87)
- ▶ There is no single best pruning algorithm

Reduced-Error Pruning

- ▶ Procedure
 - ▶ Split *training* data into *building* and *validation* sets
 - ▶ Generate an overfitting decision tree using the *building* set
 - ▶ Post-pruning: Do until further pruning is harmful:
 - ▶ Consider each of the internal non-root nodes in the tree to be candidates for pruning
 - ▶ Prune a node by removing subtree rooted at this node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with this node
 - ▶ Evaluate impact of pruning this node on the *validation* set by
 - ▶ calculating the classification error rate of the pruned tree on the *validation* set and comparing it with the error rate of the unpruned tree.
 - ▶ Greedily remove the one whose removal most reduces the error on *validation* set.
- ▶ Aim to produce smallest version of most accurate subtree
- ▶ Limitations:
 - ▶ May be sub-optimal
 - ▶ When data is limited, data used to build the tree is small

Error-Complexity Pruning

- ▶ Similar procedure to the reduced-error pruning
 - ▶ Split training data into *building* and *validation* sets
 - ▶ Generate the decision tree with the *building* set
 - ▶ Post-pruning: Do until further pruning is harmful:
 - ▶ Evaluate impact of pruning each subtree on the *validation* set
 - ▶ Greedily remove the one whose removal (resulting in tree T) minimizes the following expression on the *validation* set:

$$R_{\alpha}(T) = E(T) + \alpha L(T)$$

where $E(T)$ is the classification error of tree T , $L(T)$ is the number of leaf nodes in T and α is the **complexity** cost per leaf node.

- ▶ $R_{\alpha}(T)$ is a linear combination of the classification error rate of the tree and its complexity.
- ▶ Similar to reduced-error pruning,
 - ▶ not suitable if the size of training data is small.
 - ▶ sub-optimal

Pessimistic-Error Pruning

- ▶ This method does not require a separate validation set.
- ▶ Suppose a subtree T_s contains L leaves and J of training examples associated with T_s are misclassified by the tree
- ▶ If we replace T_s with a leaf which misclassifies E of the associated training examples, the pruned tree will be accepted if

$$E + 1/2 < J + L/2$$

- ▶ Perform top-down traversing over internal nodes
 - ▶ If an internal node is pruned, all its descendants are removed
 - ▶ Relatively fast pruning
- ▶ Advantage
 - ▶ fast: no need to separate growing and validation sets.
 - ▶ Tree building makes use of all the training data.

Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
 - Basic decision tree learning algorithm
 - How to select best attribute
 - Pruning decision tree
 - *How to deal with numeric attributes*
- Learning a regression tree
- Bagging and Boosting

How to deal with continuous attributes

Two ways:

- ▶ **Static discretization**

- ▶ Before learning a decision tree, converts a continuous attribute into a discrete attribute by partitioning the range of the continuous attribute into intervals.
- ▶ Interval labels can then be used to replace actual data values.

- ▶ **Dynamic discretization**

- ▶ Dynamically split the value range into two sub-ranges during the tree learning process

Dynamic Discretization

- ▶ Dynamically split the value range into two sub-ranges and each descendent node corresponds to a sub-range.
 - ▶ Choose to split at the middle value between two examples that are in different categories

Temperature	PlayTennis?
40	No
48	No
60	Yes
70	Yes
80	Yes
90	No

The possible split points are $\frac{48 + 60}{2} = 54$ and $\frac{80 + 90}{2} = 85$

- ▶ Evaluate the binary splitting points using the splitting criterion used for selecting attribute. For example, choose the splitting point that leads to the best information gain to compete with other attributes.

Dynamic Discretization

Example:

Temperature	PlayTennis?
40	No
48	No
60	Yes
70	Yes
80	Yes
90	No

The possible split points are $\frac{48 + 60}{2} = 54$ and $\frac{80 + 90}{2} = 85$

Possible binary splits:



Choose the one that gives the better information gain to compete with other attributes

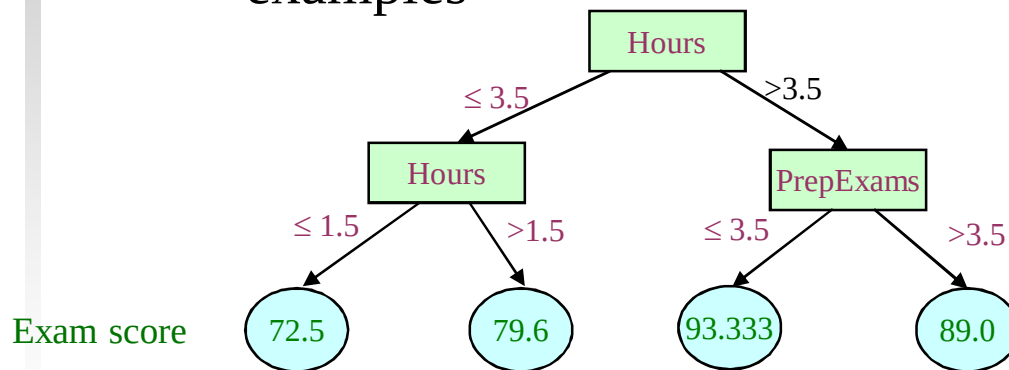
Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
 - Basic decision tree learning algorithm
 - How to select best attribute
 - Pruning decision tree
 - How to deal with numeric attributes
- *Learning a regression tree*
- Bagging and Boosting

Regression Tree

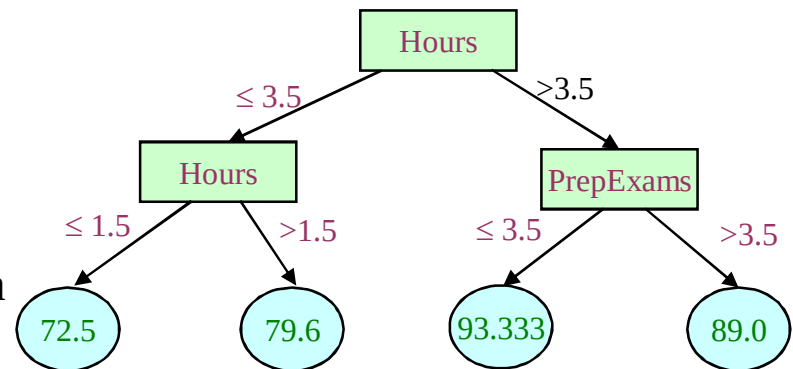
- When the target attribute is **numeric**,
 - a regression tree can be learned
 - to predict a numeric value for future examples

Hours	Prep exams	Exam score
1	1	76
2	3	78
2	3	85
4	5	88
2	2	72
1	2	69
5	1	94
4	1	94
2	1	88
4	3	92
4	4	90
3	3	75



Three Issues in Basic **Regression Tree** Induction

- ▶ How to select an attribute for a node
 - ▶ Variance (Mean Squared Error) Reduction
 - ▶ Mean Absolute Error (MAE) Reduction
- ▶ When to declare a node terminal
 - ▶ the variance is zero or less than a threshold,
 - ▶ number of examples is less than a threshold, or
 - ▶ level of the tree reaches a threshold
- ▶ How to assign a value to a leaf node
 - ▶ the mean value of examples on the target attribute

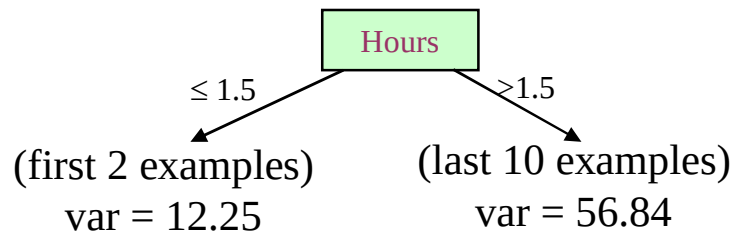


Attribute Selection Using Variance Reduction for Regression Tree Induction

- Variance of examples in a node:

$$\frac{\sum (X - \bar{X})^2}{n}$$

- Evaluate each possible split:



the weighted average of node variance in a split:

$$\frac{2}{12} \times 12.25 + \frac{10}{12} \times 56.84 = 49.41$$

- Select the split with the lowest variance

Hours	Prep exams	Exam score
1	1	76
1	2	69
2	3	78
2	3	85
2	2	72
2	1	88
3	3	75
4	5	88
4	1	94
4	3	92
4	4	90
5	1	94

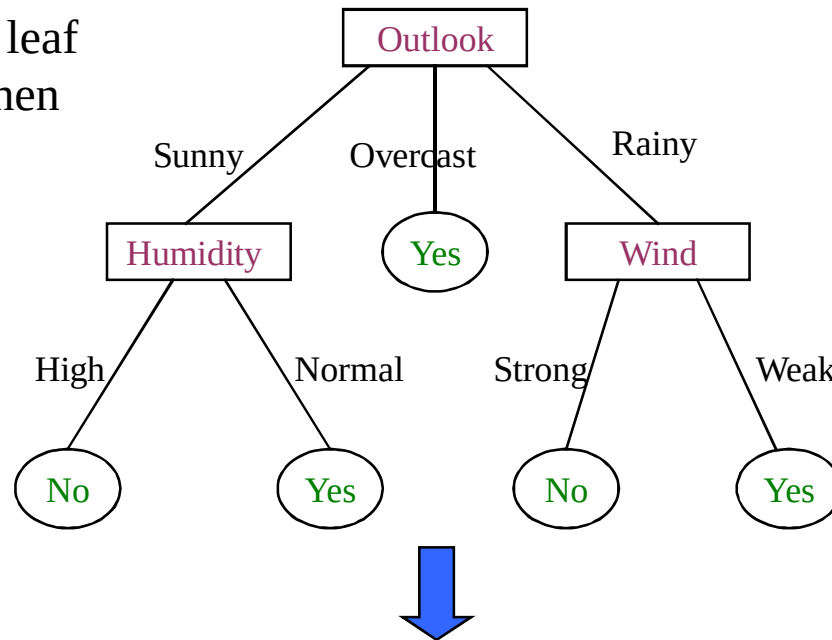


Decision Trees: Strengths

- ▶ Comprehensibility:
 - ▶ Small trees are highly interpretable and intuitive for humans
 - ▶ Can be converted into rules
- ▶ Automatic attribute selection.
- ▶ Fast classification
- ▶ Relatively fast induction
- ▶ Applicable to both regression and classification problems.
- ▶ Mature technology

Convert Decision Trees to a Set of Rules

- ▶ Each branch from the root to a leaf can be transformed into an if-then rule.



- ▶ If (Outlook is Sunny and Humidity is High), then class is No.
- ▶ If (Outlook is Sunny and Humidity is Normal), then class is Yes.
- ▶ If (Outlook is Overcast), then class is Yes.
- ▶

Decision Trees: Weaknesses

- ▶ Trees can become incomprehensible when their size grows. The large size can be caused by:
 - ▶ Rules converted from a tree
 - ▶ are *mutually exclusive*,
 - ▶ share at least one attribute (the root)
 - ▶ Thus, the size of the tree can grow much larger than the logic needed for overlapping rules (i.e., rules whose coverages overlap).
 - ▶ As an example, in a successful application of ID3 to a chess end game (Mitchie, 86), the tree representation could not be understood at all even by the chess experts.

Decision Trees: Weaknesses (*Cont'd*)

- ▶ When using only one attribute at each internal node, trees are limited to axis-parallel partitions of the instance space
- ▶ Instability
 - ▶ If we change the data a little, the tree can change a lot. So does accuracy.
 - ▶ Can be improved by building a set of trees using *Bagging* and *Boosting*
 - ▶ Random forests (Bagging)
 - ▶ Boosted trees (Boosting)
 - ▶ AdaBoost
 - ▶ Gradient boosting (e.g., XGBoost)

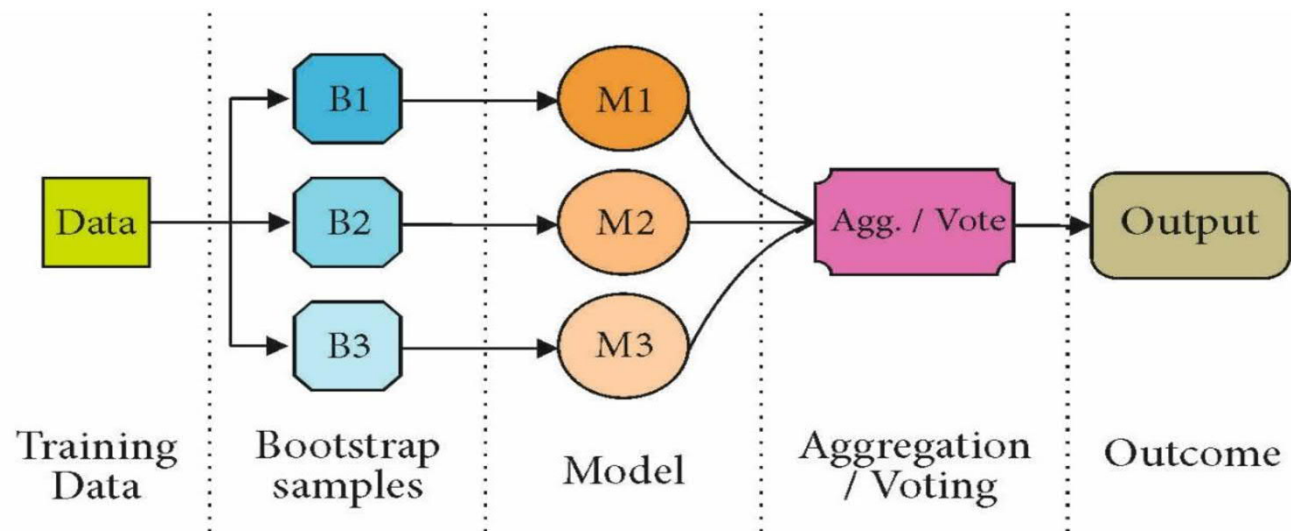
Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
 - Basic decision tree learning algorithm
 - How to select best attribute
 - Pruning decision tree
 - How to deal with numeric attributes
- Learning a regression tree
- *Bagging and Boosting*

Two General Machine Learning Approaches

- ▶ Single model approach
 - Build one really good model
- ▶ Ensemble learning
 - Build many models and average the results
 - Bagging and Boosting are two ensemble learning techniques

Bagging (Bootstrap AGGregatING)



A bootstrap sample B_i is obtained by **random sampling N examples with replacement** from the data with N examples

(Picture from <https://analyticsindiamag.com/guide-to-ensemble-methods-bagging-vs-boosting/>)

Random Forests (a Bagging Method)

- ▶ Grow a forest of many trees (e.g., 500)
- ▶ Grow each tree on an independent *bootstrap sample* from the training data with N examples
 - ▶ Bootstrap sample: Sample N examples at random with replacement
- ▶ At each node:
 - ▶ Select m attributes at random out of all M possible attributes (independently for each node).
 - ▶ Find the best split on the selected m variables.
- ▶ Grow the trees to maximum depth (i.e., no pruning)
- ▶ Vote/average the trees to get predictions for new data

Random Forests

▶ Advantages:

▶ Accuracy

- ▶ Random Forests is competitive with the best-known machine learning methods

▶ Stability in predictive performance

- ▶ If we change the data a little, the individual trees may change but the predictive performance of the forest is relatively stable because it is a combination of many trees.

▶ Disadvantage

- ▶ Longer run time than generating a single tree

- ▶ Interpretability decreases – not interpretable if there are, say, 500 trees.

Boosting Algorithms

- ▶ Principle:

- ▶ Generate **multiple weak predictors** ($f_i(x)$, $i=1, \dots, m$) and combine their predictions to form one strong prediction rule:

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \dots + w_m f_m(x)$$

- ▶ **Weak predictors** are **simple predictors that can have an accuracy slightly better than random guessing**
 - ▶ such as a **decision stump**: one-level decision tree (which uses only a single attribute for splitting).

- ▶ Use a **sequential learning strategy** to add a new base model to improve the current ensemble

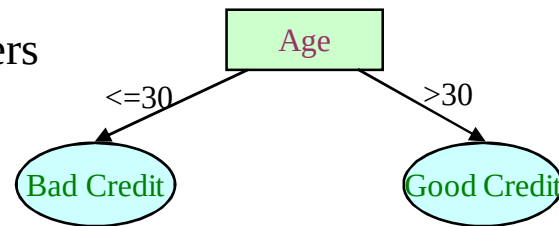
$$F_m(x) = F_{m-1}(x) + w_m f_m(x)$$

That is, learn $f_1(x)$ first, then $f_2(x)$, $f_3(x)$, and so on

Famous Boosting Algorithms

- ▶ AdaBoost (Adaptive Boosting)

- ▶ Use decision stumps as weak learners



- ▶ Adjust the weights of training examples for subsequent learners to focus on misclassified examples

- ▶ Gradient Boosting

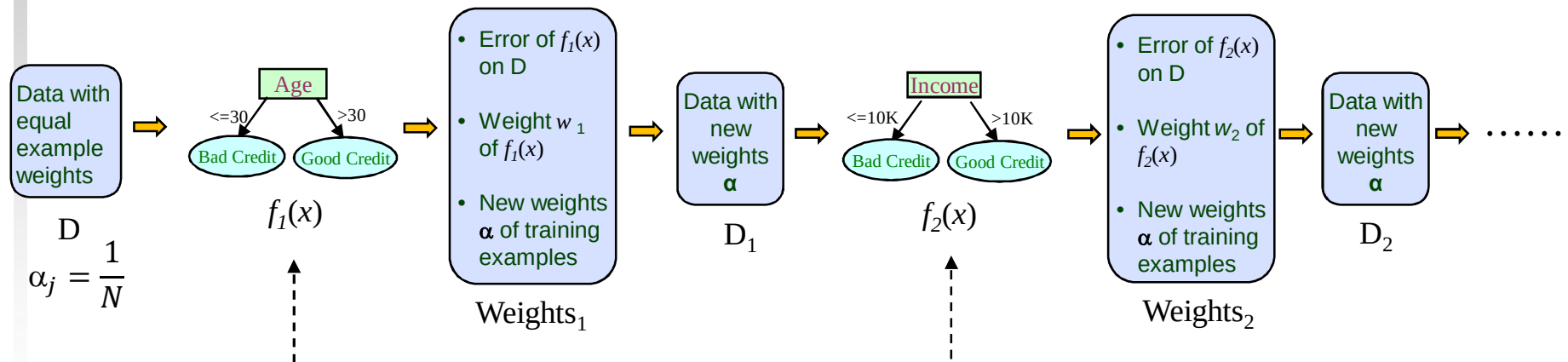
- ▶ Subsequent models learn to predict residuals (to “adjust” the previous ensemble)
 - ▶ XGBoost (eXtreme Gradient Boosting)

AdaBoost

- Learning an ensemble of weighted **binary classifiers** :

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

Each classifier $f_i(x)$
outputs -1 or 1



Choose a decision stump that minimizes the weighted classification error

How to Compute Error and Weights in AdaBoost

- ▶ Error ϵ_i of $f_i(x)$ is computed as:

$$\epsilon_i = \sum \alpha_j \text{ for all } x_j \text{ misclassified by } f_i(x)$$

which is a *weighted classification error*

- ▶ Weight w_i of $f_i(x)$ is computed as:

$$w_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right)$$

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \dots + w_m f_m(x)$$

Idea: a less accurate base model has less weight in the final classifier.

- ▶ Weight of example x_j is adjusted using:

$$\alpha_j^{new} = \begin{cases} \frac{\alpha_j^{old} \cdot e^{w_i}}{Z_i} & \text{if } x_j \text{ is misclassified by } f_i(x) \\ \frac{\alpha_j^{old} \cdot e^{-w_i}}{Z_i} & \text{if } x_j \text{ is correctly classified by } f_i(x) \end{cases}$$

Idea: If an example is misclassified, increase its weight; otherwise decrease it.

Z_i is a normalization factor (chosen so that $\sum \alpha_j^{new} = 1$).

AdaBoost

- ▶ How the weights are derived
 - ▶ Minimizing an exponential loss function
 - ▶ For details, see:
 - ▶ Yoav Freund and Robert E. Schapire, A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting:
<https://www.sciencedirect.com/science/article/pii/S002200009791504X>
- ▶ Theorem:
 - ▶ If AdaBoost generates m base models with errors $\epsilon_1, \epsilon_2, \dots, \epsilon_m$, the error of the ensemble model $F_m(x)$ is bounded as:

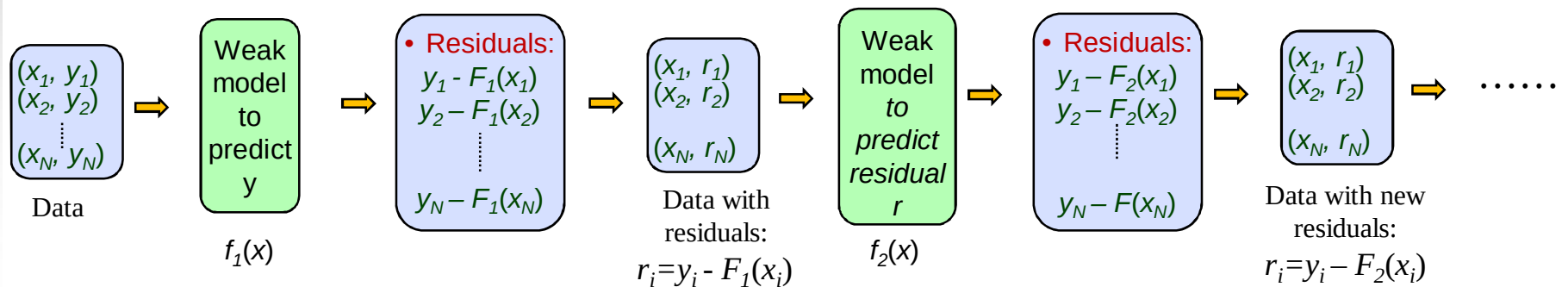
$$\epsilon \leq 2^m \prod_{i=1}^m \sqrt{\epsilon_i(1 - \epsilon_i)}$$

Gradient Boosting

- ▶ Learning an ensemble of weighted predictors (such as **regressors**):

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

- ▶ The first model $f_1(x)$ can be very weak: average of target values in the data
- ▶ Subsequent models (e.g., shallow regression trees) are built to predict residuals:



Gradient Boosting

- ▶ In gradient boosting:

$$F_m(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

w_i are learning rate, often set to a constant such as 0.1 for all the terms

- ▶ It is called “**gradient**” because training a tree on residuals is the same as training a tree on the partial derivative of the squared error loss function:

$$\frac{1}{2} (y - F_{m-1})^2$$

- ▶ Gradient boosting can be used for **both regression and classification**.

XGBoost (eXtreme Gradient Boosting)

- ▶ XGBoost (eXtreme Gradient Boosting)
 - ▶ A gradient boosting algorithm
 - ▶ Uses a more regularized model formalization to control over-fitting
 - ▶ Achieve top performance for many tasks
- ▶ For more information:
 - ▶ Video: <https://www.youtube.com/watch?v=OtD8wVaFm6E>

Summary

- Basic concepts in decision tree learning
- How to learn a decision tree
 - Basic decision tree learning algorithm
 - How to select best attribute
 - Pruning decision tree
 - How to deal with numeric attributes
- Learning a regression tree
- Bagging and Boosting
 - Random Forest, AdaBoost, Gradient boosting